**Walchand College of Engineering, Sangli**

Department of Computer Science & Engineering

**Computer Algorithm Laboratory**

**Assignment No. 4 — Greedy Method**

**Name : Atharv V Kulkarni**

**PRN : 245109074**

**Class : TY B.Tech CSE (B)**

# Question 1: Container Loading Problem

**Problem Statement:**

An airline wants to load the maximum possible number of checked-in bags into a cargo hold that has a strict weight capacity, before the flight departs. Implement the Container Loading problem to maximize the number of bags loaded without exceeding the cargo hold capacity.

**Approach:**

approach_1: try every possible subset of bags and keep the largest feasible count. tc: o(2 raised to n) sc: o(n)

approach_2 (optimal): sort bags by weight ascending and greedily add the lightest bags first until capacity is reached. tc: o(n log n) sc: o(1)

**Time Complexity:** o(n log n)

**Space Complexity:** o(1)

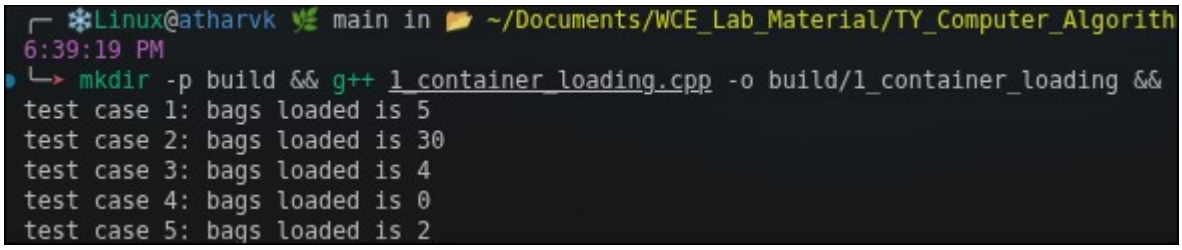
**Code:**

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
int containerLoading(vector<int>weights,int capacity){
   sort(weights.begin(),weights.end());
   int total=0,count=0;
   for(int w:weights){
      if(total+w<=capacity){
         total+=w;
```

```
        count++;
      }
      else
        break;
   }
   return count;
}
int main(){
   vector<int>test1={22,18,40,15,60,33,27,50,12,45};

vector<int>test2={22,18,40,15,60,33,27,50,12,45,38,29,55,19,42,31,24,48,36,20,58,16,47,25,34,41,13,52,28,37};
   vector<int>test3={5,10,15,20,25,30};
   vector<int>test4={1,2,3};
   vector<int>test5={100,100,100,100};
   cout<<"test case 1: bags loaded is "<<containerLoading(test1,100)<<endl;
   cout<<"test case 2: bags loaded is "<<containerLoading(test2,3000)<<endl;
   cout<<"test case 3: bags loaded is "<<containerLoading(test3,50)<<endl;
   cout<<"test case 4: bags loaded is "<<containerLoading(test4,0)<<endl;
   cout<<"test case 5: bags loaded is "<<containerLoading(test5,200)<<endl;
   return 0;
}
```

**Output:**



# Question 2: Job Sequencing with Deadlines

**Problem Statement:**

A freelance marketplace has open tasks, each with a deadline (slot number) and a payout. A freelancer can complete at most one task per day, and once a deadline passes the task can no longer be taken. Select and schedule a subset of tasks that maximizes total payout.

**Approach:**

approach_1: try every valid ordering of tasks against the deadlines and keep the best payout. tc: o(n factorial) sc: o(n)

approach_2 (optimal): sort tasks by payout descending, and place each task into the latest free slot at or before its deadline. tc: o(n squared) sc: o(n)

**Time Complexity:** o(n squared)

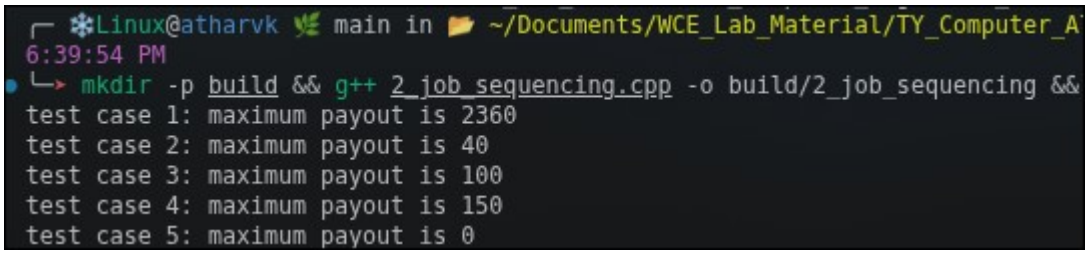
**Space Complexity:** o(n)

**Code:**

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
struct Job{
   int deadline;
   int payout;
};
bool cmpPayout(Job a,Job b){
   return a.payout>b.payout;
}
int jobSequencing(vector<Job>jobs,int slots){
   sort(jobs.begin(),jobs.end(),cmpPayout);
   vector<bool>slotUsed(slots+1,false);
   int total=0;
   for(auto job:jobs){
      for(int t=min(slots,job.deadline);t>=1;t--){
         if(!slotUsed[t]){
            slotUsed[t]=true;
            total+=job.payout;
            break;
         }
      }
   }
   return total;
}
int main(){
   vector<Job>test1={{2,450},{1,300},{3,220},{2,600},{1,150},{4,700},{3,90},{2,500},{4,260},{1,800}};
   vector<Job>test2={{1,20},{2,15},{1,10},{3,5}};
   vector<Job>test3={{1,100},{1,50}};
```

```
    vector<Job>test4={{5,10},{4,20},{3,30},{2,40},{1,50}};
    vector<Job>test5={{1,100}};
    cout<<"test case 1: maximum payout is "<<jobSequencing(test1,10)<<endl;
    cout<<"test case 2: maximum payout is "<<jobSequencing(test2,3)<<endl;
    cout<<"test case 3: maximum payout is "<<jobSequencing(test3,2)<<endl;
    cout<<"test case 4: maximum payout is "<<jobSequencing(test4,5)<<endl;
    cout<<"test case 5: maximum payout is "<<jobSequencing(test5,0)<<endl;
    return 0;
}
```

**Output:**



# Question 3: Fractional Knapsack with Indivisible Items

**Problem Statement:**

A cargo ship with a fixed hold capacity needs to be loaded with commodities, each with a total available quantity and a value. Bulk commodities can be loaded in any fraction, while manufactured goods must be loaded as a whole or not at all. Maximize the total value of cargo loaded.

**Approach:**

approach_1: try every combination of whole and fractional loadings and keep the best value. tc: o(2 raised to n) sc: o(n)

approach_2 (optimal): sort commodities by value per unit quantity descending, take whole quantities greedily, and take a fraction of the current divisible item only when capacity runs out. tc: o(n log n) sc: o(1)

**Time Complexity:** o(n log n)

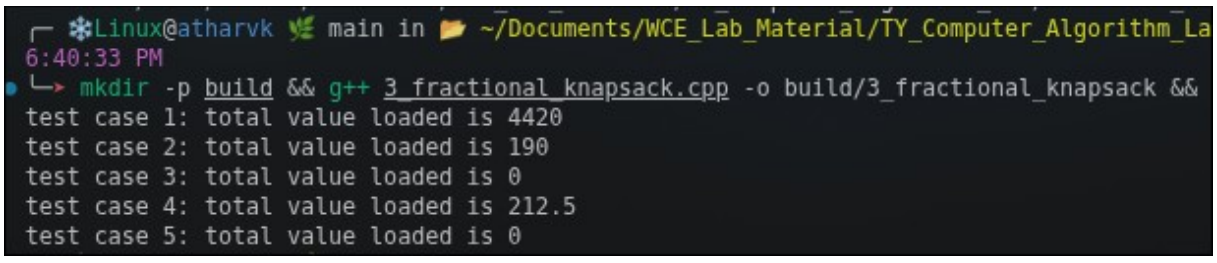
**Space Complexity:** o(1)

**Code:**

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
struct Item{
   string name;
   double qty;
   double value;
   bool indivisible;
};
bool cmpRatio(Item a,Item b){
   return (a.value/a.qty)>(b.value/b.qty);
}
double knapsack(vector<Item>items,double capacity){
   sort(items.begin(),items.end(),cmpRatio);
   double total=0,remaining=capacity;
   for(auto item:items){
      if(remaining<=0)
         break;
      if(item.qty<=remaining){
         total+=item.value;
         remaining-=item.qty;
      }
      else if(!item.indivisible){
         double fraction=remaining/item.qty;
         total+=item.value*fraction;
         remaining=0;
      }
   }
```

```
    return total;
}
int main(){
    vector<Item>test1={
        {"crudeoil",300,900,false},
        {"coal",250,700,false},
        {"machinery",150,650,true},
        {"grain",200,500,false},
        {"steel",180,620,true},
        {"chemicals",100,450,false},
        {"vehicles",120,800,true},
        {"textiles",90,300,false},
        {"electronics",60,700,true},
        {"fertilizer",140,400,false}
    };
    vector<Item>test2={{"a",20,100,false},{"b",30,90,false}};
    vector<Item>test3={{"x",15,150,true}};
    vector<Item>test4={{"p",10,100,true},{"q",20,150,false}};
    vector<Item>test5={{"z",5,50,false}};
    cout<<"test case 1: total value loaded is "<<knapsack(test1,1000)<<endl;
    cout<<"test case 2: total value loaded is "<<knapsack(test2,50)<<endl;
    cout<<"test case 3: total value loaded is "<<knapsack(test3,10)<<endl;
    cout<<"test case 4: total value loaded is "<<knapsack(test4,25)<<endl;
    cout<<"test case 5: total value loaded is "<<knapsack(test5,0)<<endl;
    return 0;
}
```

**Output:**



# Question 4: Optimal Merge Pattern

**Problem Statement:**

A hospital network needs to merge patient record files arriving from branch clinics into one master file. Files can only be merged two at a time, and the cost of merging two files equals the sum of their sizes. Determine the merge order that minimizes the total merge cost.

**Approach:**

approach_1: try every possible order of pairwise merges and keep the cheapest. tc: o(n factorial) sc: o(n)

approach_2 (optimal): repeatedly pull the two smallest files from a min-heap, merge them, and push the merged size back until one file remains. tc: o(n log n) sc: o(n)

**Time Complexity:** o(n log n)

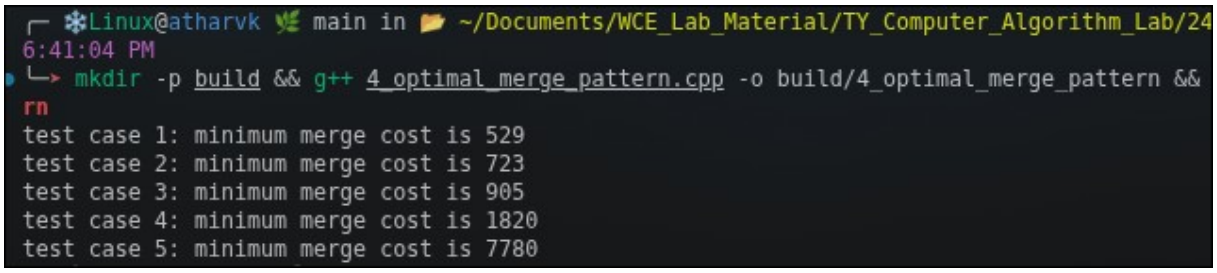
**Space Complexity:** o(n)

**Code:**

```
#include<iostream>
#include<vector>
#include<queue>
using namespace std;
int optimalMerge(vector<int>sizes){
   priority_queue<int,vector<int>,greater<int>>pq(sizes.begin(),sizes.end());
   int totalcost=0;
   while(pq.size()>1){
      int a=pq.top();
      pq.pop();
      int b=pq.top();
      pq.pop();
      int merged=a+b;
      totalcost+=merged;
      pq.push(merged);
   }
   return totalcost;
}
int main(){
   vector<int>branchA={35,110,180,8};
   vector<int>branchB={25,12,240,85,55};
   vector<int>branchC={95,65,30,50,80,40};
   vector<int>branchD={480,290,140,20,65};
   vector<int>allBranches={35,110,180,8,25,12,240,85,55,95,65,30,50,80,40,480,290,140,20,65};
   cout<<"test case 1: minimum merge cost is "<<optimalMerge(branchA)<<endl;
   cout<<"test case 2: minimum merge cost is "<<optimalMerge(branchB)<<endl;
   cout<<"test case 3: minimum merge cost is "<<optimalMerge(branchC)<<endl;
   cout<<"test case 4: minimum merge cost is "<<optimalMerge(branchD)<<endl;
   cout<<"test case 5: minimum merge cost is "<<optimalMerge(allBranches)<<endl;
   return 0;
```

```
}
```

**Output:**


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/24
6:41:04 PM
mkdir -p build && g++ 4_optimal_merge_pattern.cpp -o build/4_optimal_merge_pattern &&
rn
test case 1: minimum merge cost is 529
test case 2: minimum merge cost is 723
test case 3: minimum merge cost is 905
test case 4: minimum merge cost is 1820
test case 5: minimum merge cost is 7780

# Question 5: Huffman Coding

**Problem Statement:**

A satellite transmits telemetry data from onboard instrument channels back to ground control. Each channel has a transmission frequency and a priority factor; channels that are both frequent and high-priority should receive the shortest possible codes. Implement Huffman Coding to generate optimal prefix codes for the telemetry channels.

**Approach:**

approach_1: enumerate every possible binary prefix code tree over the channels and keep the one with the lowest weighted length. tc: o(2 raised to n) sc: o(n)

approach_2 (optimal): repeatedly merge the two lowest-weight nodes from a min-heap into a new tree node until one root remains, then read off codes by walking left as 0 and right as 1. tc: o(n log n) sc: o(n)

**Time Complexity:** o(n log n)
**Space Complexity:** o(n)

**Code:**

```
#include<iostream>
#include<vector>
#include<queue>
#include<map>
using namespace std;
struct Node{
   string name;
   long long weight;
   Node*left;
   Node*right;
};
struct cmpNode{
   bool operator()(Node*a,Node*b){
      return a->weight>b->weight;
   }
};
void generateCodes(Node*root,string code,map<string,string>&codes){
   if(root==NULL)
      return;
   if(root->left==NULL&&root->right==NULL){
      codes[root->name]=(code.empty())?"0":code;
      return;
   }
   generateCodes(root->left,code+"0",codes);
   generateCodes(root->right,code+"1",codes);
}
void runCase(vector<pair<string,long long>>channels){
   priority_queue<Node*,vector<Node*>,cmpNode>pq;
```

```
    for(auto ch:channels){
        Node*n=new Node();
        n->name=ch.first;
        n->weight=ch.second;
        n->left=NULL;
        n->right=NULL;
        pq.push(n);
    }
    while(pq.size()>1){
        Node*a=pq.top();
        pq.pop();
        Node*b=pq.top();
        pq.pop();
        Node*merged=new Node();
        merged->name="";
        merged->weight=a->weight+b->weight;
        merged->left=a;
        merged->right=b;
        pq.push(merged);
    }
    Node*root=pq.top();
    map<string,string>codes;
    generateCodes(root,"",codes);
    long long totalweight=0,weightedlen=0;
    string shortest="",longestc="";
    for(auto ch:channels){
        string code=codes[ch.first];
        cout<<ch.first<<"="<<code<<" ";
        totalweight+=ch.second;
        weightedlen+=ch.second*code.size();
        if(shortest==""||code.size()<codes[shortest].size())
            shortest=ch.first;
        if(longestc==""||code.size()>codes[longestc].size())
            longestc=ch.first;
    }
    cout<<endl;
    cout<<"shortest code is "<<shortest<<"="<<codes[shortest]<<" longest code is
"<<longestc<<"="<<codes[longestc]<<endl;
    double avg=(double)weightedlen/totalweight;
    cout<<"average code length is "<<avg<<endl;
}
int main(){
    vector<pair<string,long long>>test1={
        {"temperature",380},{"pressure",560},{"voltage",300},{"gyro",150},{"thruster",110},
        {"attitudecontrol",190},{"solarpanel",240},{"batterylevel",420},{"communication",500},
        {"radiation",70},{"magnetometer",60},{"startracker",200},{"reactionwheel",170},
        {"fuellevel",130},{"antenna",90},{"camera",210},{"gps",85},{"accelerometer",65},
        {"databus",150},{"timing",100}
```

```
    };
    vector<pair<string,long long>>test2={{"a",5},{"b",9},{"c",12},{"d",13},{"e",16},{"f",45}};
    vector<pair<string,long long>>test3={{"a",1},{"b",1},{"c",1},{"d",1}};
    vector<pair<string,long long>>test4={{"a",10},{"b",20}};
    vector<pair<string,long long>>test5={{"a",1},{"b",2},{"c",4},{"d",8},{"e",16}};
    cout<<"test case 1:"<<endl;
    runCase(test1);
    cout<<"test case 2:"<<endl;
    runCase(test2);
    cout<<"test case 3:"<<endl;
    runCase(test3);
    cout<<"test case 4:"<<endl;
    runCase(test4);
    cout<<"test case 5:"<<endl;
    runCase(test5);
    return 0;
}
```

**Output:**

```
┌ ❄Linux@atharvk 🌿main in 📂~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_4
42:08 PM
└➤ mkdir -p build && g++ 5_huffman_coding.cpp -o build/5_huffman_coding && ./build/5_huffman_coding
shortest code is pressure=101 longest code is radiation=110110
average code length is 4.04067
test case 2:
a=1100 b=1101 c=100 d=101 e=111 f=0
shortest code is f=0 longest code is a=1100
average code length is 2.24
test case 3:
a=00 b=10 c=01 d=11
shortest code is a=00 longest code is a=00
average code length is 2
test case 4:
a=0 b=1
shortest code is a=0 longest code is a=0
average code length is 1
test case 5:
a=0000 b=0001 c=001 d=01 e=1
shortest code is e=1 longest code is a=0000
average code length is 1.80645
```